

Algorytmy Zaawansowane

Rozwiązania egzaminów z lat ubiegłych

Rozwiązujemy zadania wymagające pracy; te będące powtórzeniem materiału z wykładu odsyłają do notatek.

Spis treści

1 Termin 1	2
2 Termin 2	3
3 Termin 3	4
A Termin 1	5
B Termin 2	9
C Termin 3	13

Termin 1

Zadanie 1 — przecinające się kwadraty jednostkowe (15 p.)

Zaprojektuj rekurencyjny algorytm w oparciu o strategię „dziel i zwyciężaj” rozwiązujący następujący problem. *Dane:* zbiór kwadratów na płaszczyźnie o boku jeden, których boki są równoległe do osi. *Szukane:* para przecinających się kwadratów lub odpowiedź, że takiej pary nie ma. Oblicz złożoność obliczeniową zaprojektowanego algorytmu.

Zadanie 2 — liczba etykietowań $L(2,0)$ ścieżki (15 p.)

Etykietowaniem $L(2,0)$ grafu G nazywamy funkcję przyporządkowującą wierzchołkom G liczby ze zbioru $\{1, 2, \dots, k\}$ taką, że sąsiednie wierzchołki otrzymują etykiety różniące się o co najmniej 2. Zaprojektuj wielomianowy algorytm programowania dynamicznego zliczający liczbę etykietowań $L(2,0)$ czterema etykietami ($k = 4$) dla ścieżki o n wierzchołkach. Oszacuj asymptotyczną złożoność (w terminach $\mathcal{O}(\cdot)$).

Zadanie 3 — najmniejsza transversala (15 p.)

Niech $\mathcal{S}$ będzie rodziną pewnych podzbiorów zbioru U . *Transversalą* dla rodziny $\mathcal{S}$ nazywamy dowolny podzbiór U przecinający każdy zbiór z $\mathcal{S}$. Podaj algorytm aproksymacyjny dla problemu znajdowania najmniejszej transversali przy założeniu, że każdy element zbioru U należy do co najwyżej dwóch zbiorów z $\mathcal{S}$. *Wskazówka:* sprowadź ten problem do problemu pokrycia zbioru.

Zadanie 4 — ze schematu prawie-optimalnego do 2-aproksymacji (15 p.)

Niech P będzie problemem optymalizacyjnym, w którym poszukujemy rozwiązania dopuszczalnego maksymalizującego funkcję celu f , której zbiorem wartości jest przedział $[3, \infty)$. Wiadomo, że dla problemu P i każdego $\epsilon > 0$ istnieje algorytm wielomianowy, który dla wejścia I znajduje rozwiązanie dopuszczalne $A(I)$ takie, że $f(A(I)) \geq (1 - \epsilon)f(OPT(I)) - 1$, gdzie $OPT(I)$ to rozwiązanie optymalne. Udowodnij, że istnieje wielomianowy algorytm 2-aproksymacyjny dla problemu P .

Termin 2

Zadanie 1 — scalanie list (15 p.)

- (i) (3p.) Niech A, B będą listami długości k i l posortowanymi rosnąco. Podaj algorytm działający w czasie $\mathcal{O}(k+l)$, który scala je w jedną posortowaną rosnąco listę długości $k+l$.
- (ii) (4p.) Mamy m list długości n każda, wszystkie posortowane rosnąco; scalamy je w jedną długości mn , używając wielokrotnie algorytmu z (i): scalamy dwie pierwsze listy, wynik z trzecią, ten z czwartą, itd. Oszacuj złożoność jako funkcję m i n .
- (iii) (8p.) Podaj asymptotycznie szybszy algorytm typu „dziel i zwyciężaj” rozwiązujący problem z (ii). Oszacuj jego złożoność.

Zadanie 2 — brak słów kodowych długości 1 (15 p.)

Załóżmy, że w tekście, który chcemy zakodować za pomocą kodu Huffmana, każdy znak występuje mniej niż $\frac{24}{3}$ razy. Udowodnij, że w kodzie Huffmana dla takiego tekstu żaden znak nie będzie zakodowany za pomocą tylko jednego bitu.

Zadanie 3 — pakowanie do skrzyń (Next-Fit) jest 2-aproksymacyjne (15 p.)

Rozważmy następujący algorytm dla problemu pakowania do skrzyń. Wejściem jest nieposortowana lista rozmiarów pakowanych obiektów. Algorytm pakuje obiekty w kolejności z listy, próbując umieścić kolejny obiekt tylko w ostatnio rozpoczętej skrzyni; jeśli to niemożliwe, rozpoczyna nową skrzynię. Udowodnij, że jest to algorytm 2-aproksymacyjny dla problemu pakowania do skrzyń.

Zadanie 4 — najdłuższy wspólny podciąg kolejnych wyrazów (15 p.)

Dla zadanych ciągów $(x_1, x_2, \dots, x_n)$ i $(y_1, y_2, \dots, y_m)$ chcemy znaleźć długość ich najdłuższego wspólnego podciągu *kolejnych* wyrazów, czyli największe k , dla którego istnieją indeksy i, j takie, że $(x_i, x_{i+1}, \dots, x_{i+k-1}) = (y_j, y_{j+1}, \dots, y_{j+k-1})$. Podaj algorytm programowania dynamicznego o złożoności $\mathcal{O}(mn)$ rozwiązujący ten problem.

Termin 3

Zadanie 1 — kod Huffmana dla częstości będących potęgami dwójki (15 p.)

Alfabet A składa się ze znaków $z_1, z_2, \dots, z_n$. W pewnym tekście T znak z_i występuje 2^{i-1} razy dla wszystkich $i \in \{1, 2, \dots, n\}$. Znajdź kod Huffmana dla tekstu T .

Zadanie 2 — rozdział czekolady (15 p.)

Kostka czekolady o wadze $2n$ dkg spadła na podłogę i połamano się na wiele kawałków, z których żaden nie waży więcej niż 1 dkg. Zaprojektuj algorytm o liniowej złożoności obliczeniowej, który rozdzieli te kawałki pomiędzy n dzieci tak, że każde dziecko dostaje kawałki o sumarycznej wadze co najmniej 1 dkg. Udowodnij, że Twój algorytm jest poprawny.

Zadanie 3 — stacje przekaźnikowe (DP) (15 p.)

Firma „Głuchy Telefon” buduje stacje przekaźnikowe wzdłuż autostrady A44 o długości M km. Jest n możliwych lokalizacji, leżących w odległościach $m_1 < m_2 < \dots < m_n$ od początku autostrady (wszystkie przy autostradzie). Koszt budowy stacji w lokalizacji i -tej to $k_i > 0$, $i \in \{1, \dots, n\}$. Stacje trzeba wybudować tak, by z każdego punktu autostrady było nie dalej niż k km do najbliższej stacji. Podaj efektywny algorytm programowania dynamicznego obliczający najmniejszy całkowity koszt inwestycji i oszacuj jego złożoność obliczeniową.

Zadanie 4 — obiekt majoryzujący (dziel i zwyciężaj) (15 p.)

Tablica A zawiera n obiektów, które nie są liczbami (np. małe pliki tekstowe). Nie jest dla nich zdefiniowana relacja $>$, ale zdefiniowana jest równość (sprawdzana w czasie stałym), oznaczająca identyczność obiektów. Mówimy, że tablica A ma *obiekt majoryzujący* M , jeśli więcej niż połowa obiektów jest równa M . Podaj algorytm typu „dziel i zwyciężaj” o złożoności $\mathcal{O}(n \log n)$, który rozstrzyga, czy A zawiera obiekt majoryzujący, i jeśli tak — zwraca go.

Termin 1

Zadanie 1 — przecinające się kwadraty jednostkowe (15 p.)

Zaprojektuj rekurencyjny algorytm w oparciu o strategię „dziel i zwyciężaj” rozwiązujący następujący problem. *Dane:* zbiór kwadratów na płaszczyźnie o boku jeden, których boki są równoległe do osi. *Szukane:* para przecinających się kwadratów lub odpowiedź, że takiej pary nie ma. Oblicz złożoność obliczeniową zaprojektowanego algorytmu.

Sprowadzenie do pary najbliższych punktów. Każdy kwadrat o boku 1 i bokach równoległych do osi reprezentujemy jego środkiem $p_i = (x_i, y_i)$. Dwa takie kwadraty p_i, p_j przecinają się (mają wspólny punkt) wtedy i tylko wtedy, gdy nakładają się ich rzuty na obie osie:

$$|x_i - x_j| \leq 1 \quad \text{oraz} \quad |y_i - y_j| \leq 1, \quad \text{czyli} \quad d_\infty(p_i, p_j) = \max(|x_i - x_j|, |y_i - y_j|) \leq 1.$$

(Dla samych wnętrz zamieniamy $\leq$ na $<$; nie zmienia to algorytmu.) Szukamy więc pary punktów w odległości $d_\infty \leq 1$ albo stwierdzamy, że jej nie ma. Wystarczy znaleźć parę najbliższą w metryce d_∞ i porównać jej odległość z 1: jeśli minimalna odległość > 1 , to żadna para się nie przecina.

Algorytm. Stosujemy schemat „dziel i zwyciężaj” dla pary najbliższych punktów (notatki, wykład o parze najbliższych punktów: rys. 1), zmieniając jedynie metrykę z euklidesowej na d_∞ :

- **Dziel.** Sortujemy punkty po x (raz, wstępnie) i dzielimy pionową prostą l na dwie połowy po $\lfloor n/2 \rfloor$ punktów.
- **Zwyciężaj.** Rekurencyjnie wyznaczamy najmniejsze δ_L, δ_R w obu połowach; $\delta = \min(\delta_L, \delta_R)$.
- **Połącz.** W pasie $|x - l| \leq \delta$ (punkty posortowane po y) każdy punkt porównujemy tylko z następnymi, których współrzędna y różni się o $\leq \delta$. Jak w dowodzie dla pary najbliższych punktów, w prostokącie $2\delta \times \delta$ leży $\mathcal{O}(1)$ punktów (gdyby było ich więcej, dwa leżałyby w odległości $d_\infty < \delta$ po tej samej stronie — sprzeczność z δ), więc faza łączenia kosztuje $\mathcal{O}(n)$.

Dla $n \leq 3$ liczymy wszystkie pary wprost. Na końcu zwracamy najbliższą parę, gdy jej $d_\infty \leq 1$, w przeciwnym razie „brak pary”.

Złożoność. Wstępne sortowanie $\mathcal{O}(n \log n)$; rekurencja $T(n) = 2T(n/2) + \mathcal{O}(n) = \mathcal{O}(n \log n)$. Łącznie $\boxed{\mathcal{O}(n \log n)}$.

Zadanie 2 — liczba etykietowań $L(2,0)$ ścieżki (15 p.)

Etykietowaniem $L(2,0)$ grafu G nazywamy funkcję przyporządkowującą wierzchołkom G liczby ze zbioru $\{1, 2, \dots, k\}$ taką, że sąsiednie wierzchołki otrzymują etykiety różniące się o co najmniej 2. Zaprojektuj wielomianowy algorytm programowania dynamicznego zliczający liczbę etykietowań $L(2,0)$ czterema etykietami ($k = 4$) dla ścieżki o n wierzchołkach. Oszacuj asymptotyczną złożoność (w terminach $\mathcal{O}(\cdot)$).

Etykietujemy wierzchołki ścieżki $v_1, \dots, v_n$ liczbami ze zbioru $\{1, 2, 3, 4\}$ tak, że sąsiednie etykiety różnią się o co najmniej 2. Dwie etykiety a, b są zgodne, gdy $|a - b| \geq 2$; zgodne pary (nieuporządkowane) to

$$\{1, 3\}, \{1, 4\}, \{2, 4\} \quad (\text{oraz } a = b \text{ jest zawsze niezgodne}).$$

Programowanie dynamiczne. Niech $D[i][c]$ to liczba poprawnych etykietowań prefiksu $v_1, \dots, v_i$, w których v_i ma etykietę c . Wtedy

$$D[1][c] = 1 \quad (c = 1, \dots, 4), \quad D[i][c] = \sum_{\substack{c'=1 \\ |c-c'| \geq 2}}^4 D[i-1][c'],$$

a odpowiedzią jest $\sum_{c=1}^4 D[n][c]$. Tablicę wypełniamy dla $i = 2, \dots, n$; każda komórka to suma po ≤ 4 poprzednikach.

Złożoność. n wierszy $\times 4$ etykiety $\times \mathcal{O}(1)$ pracy na komórkę: $\boxed{\mathcal{O}(n)}$ czasu i $\mathcal{O}(1)$ pamięci (trzymamy tylko wiersz $i - 1$).

Zadanie 3 — najmniejsza transversala (15 p.)

Niech $\mathcal{S}$ będzie rodziną pewnych podzbiorów zbioru U . *Transwersalą* dla rodziny $\mathcal{S}$ nazywamy dowolny podzbiór U przecinający każdy zbiór z $\mathcal{S}$. Podaj algorytm aproksymacyjny dla problemu znajdowania najmniejszej transversali przy założeniu, że każdy element zbioru U należy do co najwyżej dwóch zbiorów z $\mathcal{S}$. *Wskazówka:* sprowadź ten problem do problemu pokrycia zbioru.

Dana rodzina $\mathcal{S}$ podzbiorów zbioru U ; transwersala to $T \subseteq U$ przecinająca każdy zbiór z $\mathcal{S}$. Zakładamy, że każdy element U należy do co najwyżej dwóch zbiorów z $\mathcal{S}$.

Sprowadzenie do pokrycia zbioru. Tworzymy instancję pokrycia zbioru, w której pokrywamy rodzinę $\mathcal{S}$: zbiorem bazowym jest $X = \mathcal{S}$, a dla każdego elementu $u \in U$ definiujemy zbiór pokrywający

$$S_u = \{ A \in \mathcal{S} : u \in A \} \subseteq \mathcal{S}.$$

Wybranie elementu u do transversali „pokrywa” dokładnie te A , które u przecina. Zatem $\{u_1, \dots, u_t\}$ jest transversalą wtedy i tylko wtedy, gdy $S_{u_1}, \dots, S_{u_t}$ pokrywają $X = \mathcal{S}$, przy równym koszcie t . Najmniejsza transversala = najmniejsze pokrycie tej instancji.

Algorytm i współczynnik. Założenie „każdy u w co najwyżej dwóch zbiorach” daje $|S_u| \leq 2$ dla każdego u . Uruchamiamy zachłanny algorytm pokrycia zbioru (GREEDY-SET-COVER, Algorytm 17). Z twierdzenia o jego jakości jest on H_d -aproksymacyjny, gdzie $d = \max_u |S_u|$ to liczność największego zbioru pokrywającego. Tu $d \leq 2$, więc współczynnik wynosi

$$H_2 = 1 + \frac{1}{2} = \frac{3}{2},$$

a zatem otrzymujemy algorytm $\frac{3}{2}$ -aproksymacyjny (w szczególności 2-aproksymacyjny) dla najmniejszej transversali.

Zadanie 4 — ze schematu prawie-optimalnego do 2-aproksymacji (15 p.)

Niech P będzie problemem optymalizacyjnym, w którym poszukujemy rozwiązania dopuszczalnego maksymalizującego funkcję celu f , której zbiorem wartości jest przedział $[3, \infty)$. Wiadomo, że dla problemu P i każdego $\epsilon > 0$ istnieje algorytm wielomianowy, który dla wejścia I znajduje rozwiązanie dopuszczalne $A(I)$ takie, że $f(A(I)) \geq (1 - \epsilon)f(OPT(I)) - 1$, gdzie $OPT(I)$ to rozwiązanie optymalne. Udowodnij, że istnieje wielomianowy algorytm 2-aproksymacyjny dla problemu P .

Dla każdego $\epsilon > 0$ mamy wielomianowy A_ϵ z gwarancją $f(A_\epsilon(I)) \geq (1 - \epsilon)f(OPT(I)) - 1$, a wartości f leżą w $[3, \infty)$, więc $f(OPT(I)) \geq 3$ dla każdej instancji. Chcemy algorytmu 2-aproksymacyjnego, tj. $f(A(I)) \geq \frac{1}{2}f(OPT(I))$.

Konstrukcja. Ustalamy $\epsilon = \frac{1}{6}$ (stała, więc $A_{1/6}$ pozostaje wielomianowy). Oznaczmy $V = f(OPT(I)) \geq 3$. Wtedy

$$f(A_{1/6}(I)) \geq \left(1 - \frac{1}{6}\right)V - 1 = \frac{5}{6}V - 1 = \frac{1}{2}V + \left(\frac{1}{3}V - 1\right) \geq \frac{1}{2}V,$$

bo $\frac{1}{3}V - 1 \geq 0 \iff V \geq 3$, co zachodzi. Zatem $A_{1/6}$ jest wielomianowym algorytmem 2-aproksymacyjnym dla P . $\square$

Termin 2

Zadanie 1 — scalanie list (15 p.)

- (i) (3p.) Niech A, B będą listami długości k i l posortowanymi rosnąco. Podaj algorytm działający w czasie $\mathcal{O}(k+l)$, który scala je w jedną posortowaną rosnąco listę długości $k+l$.
- (ii) (4p.) Mamy m list długości n każda, wszystkie posortowane rosnąco; scalamy je w jedną długości mn , używając wielokrotnie algorytmu z (i): scalamy dwie pierwsze listy, wynik z trzecią, ten z czwartą, itd. Oszacuj złożoność jako funkcję m i n .
- (iii) (8p.) Podaj asymptotycznie szybszy algorytm typu „dziel i zwyciężaj” rozwiązujący problem z (ii). Oszacuj jego złożoność.

(i) **Scalanie dwóch list w czasie $\mathcal{O}(k+l)$.** Klasyczne scalanie dwoma wskaźnikami (procedura MERGE z sortowania przez scalanie; por. MERGELISTS z notatek o sumie podzbiorów, Algorytm 18):

Algorytm 1: Merge — scalanie dwóch list posortowanych

```

1: function MERGE( $A[1 \dots k], B[1 \dots l]$ )
2:    $i \leftarrow 1; j \leftarrow 1; C \leftarrow$  pusta lista
3:   while  $i \leq k$  and  $j \leq l$  do
4:     if  $A[i] \leq B[j]$  then
5:       dołącz  $A[i]$  do  $C$ ;  $i \leftarrow i + 1$ 
6:     else
7:       dołącz  $B[j]$  do  $C$ ;  $j \leftarrow j + 1$ 
8:   dołącz do  $C$  pozostałe elementy  $A$  lub  $B$ 
9:   return  $C$ 

```

Każda iteracja przesuwają o jeden któryś ze wskaźników, więc czas $\mathcal{O}(k+l)$.

(ii) **Scalanie iteracyjne.** Scalamy listy po kolei: wynik długości n z kolejną listą długości n , potem długości $2n$ z następną, itd. Scalenie przed dołączeniem i -tej listy ma rozmiar $(i-1)n$, więc kosztuje $\mathcal{O}((i-1)n + n) = \mathcal{O}(in)$. Łącznie

$$\sum_{i=2}^m \mathcal{O}(in) = \mathcal{O}\left(n \sum_{i=2}^m i\right) = \boxed{\mathcal{O}(m^2 n)}.$$

(iii) **Scalanie „dziel i zwyciężaj”.** Scalamy listy parami (poziom 1: $m/2$ scaleń list długości n), potem powstałe pary parami, itd. — jak w sortowaniu przez scalanie po listach jako liściach. Na każdym z $\lceil \log_2 m \rceil$ poziomów łączna długość scalanych danych wynosi mn , więc poziom kosztuje $\mathcal{O}(mn)$. Stąd

$$T(m, n) = \boxed{\mathcal{O}(mn \log m)},$$

asymptotycznie szybciej niż $\mathcal{O}(m^2 n)$ z podpunktu (ii).

Zadanie 2 — brak słów kodowych długości 1 (15 p.)

Załóżmy, że w tekście, który chcemy zakodować za pomocą kodu Huffmana, każdy znak występuje mniej niż $\frac{24}{3}$ razy. Udowodnij, że w kodzie Huffmana dla takiego tekstu żaden znak nie będzie zakodowany za pomocą tylko jednego bitu.

Lemat B.1. *Jeśli w tekście o łącznej liczbie wystąpień T pewien znak x otrzymuje w kodzie Huffmana słowo długości 1, to $f(x) \geq T/3$.*

Dowód. Słowo długości 1 znaczy, że liść x jest dzieckiem korzenia. Drugim poddrzewem korzenia jest R obejmujące wszystkie pozostałe znaki, o wadze $f(R) = T - f(x)$. Przy ≥ 3 znakach R jest węzłem wewnętrznym, powstałym ze scalenia dwóch węzłów o wagach a, b z $a + b = T - f(x)$. W kroku, w którym powstał korzeń R , węzeł x był jeszcze dostępny, a mimo to scalono a i b — czyli były to dwa najlżejsze węzły, więc $f(x) \geq a$ oraz $f(x) \geq b$. Zatem

$$f(x) \geq \max(a, b) \geq \frac{a+b}{2} = \frac{T-f(x)}{2} \implies 3f(x) \geq T \implies f(x) \geq \frac{T}{3}.$$

□

Wniosek (teza zadania). Jeśli każdy znak występuje mniej niż $T/3$ razy (np. mniej niż $\frac{24}{3} = 8$ razy w tekście długości $T = 24$), to żaden znak nie spełnia $f(x) \geq T/3$, więc — na mocy lematu — żaden znak nie otrzyma słowa kodowego długości 1. □

Zadanie 3 — pakowanie do skrzyń (Next-Fit) jest 2-aproksymacyjne (15 p.)

Rozważmy następujący algorytm dla problemu pakowania do skrzyń. Wejściem jest nieposortowana lista rozmiarów pakowanych obiektów. Algorytm pakuje obiekty w kolejności z listy, próbując umieścić kolejny obiekt tylko w ostatnio rozpoczętej skrzyni; jeśli to niemożliwe, rozpoczyna nową skrzynię. Udowodnij, że jest to algorytm 2-aproksymacyjny dla problemu pakowania do skrzyń.

Skrzynie mają pojemność 1, rozmiary obiektów $s_1, \dots, s_n \in (0, 1]$. Algorytm (*Next-Fit*) pakuje obiekt do ostatnio rozpoczętej skrzyni, a gdy się nie mieści — rozpoczyna nową. Niech $S = \sum_i s_i$ i niech algorytm użyje k skrzyń, a optimum OPT .

Dolne ograniczenie optimum. Żadne upakowanie nie zmieści więcej niż 1 w skrzyni, więc $OPT \geq \lceil S \rceil \geq S$.

Górne ograniczenie dla Next-Fit. Rozważmy dwie kolejne skrzynie B_i, B_{i+1} . Pierwszy obiekt trafiający do B_{i+1} nie zmieścił się w B_i , więc suma zawartości obu skrzyń przekracza pojemność: $\text{cont}(B_i) + \text{cont}(B_{i+1}) > 1$ dla $i = 1, \dots, k-1$. Sumując te nierówności i korzystając z $\sum_i \text{cont}(B_i) = S$:

$$2S = \sum_{i=1}^k 2 \text{cont}(B_i) \geq \sum_{i=1}^{k-1} (\text{cont}(B_i) + \text{cont}(B_{i+1})) > k - 1.$$

Stąd $k < 2S + 1$, a ponieważ k jest całkowite, $k \leq 2S \leq 2OPT$.

Wniosek. $k \leq 2OPT$, więc Next-Fit jest algorytmem 2-aproksymacyjnym. $\square$

Zadanie 4 — najdłuższy wspólny podciąg kolejnych wyrazów (15 p.)

Dla zadanych ciągów $(x_1, x_2, \dots, x_n)$ i $(y_1, y_2, \dots, y_m)$ chcemy znaleźć długość ich najdłuższego wspólnego podciągu *kolejnych* wyrazów, czyli największe k , dla którego istnieją indeksy i, j takie, że $(x_i, x_{i+1}, \dots, x_{i+k-1}) = (y_j, y_{j+1}, \dots, y_{j+k-1})$. Podaj algorytm programowania dynamicznego o złożoności $\mathcal{O}(mn)$ rozwiązujący ten problem.

Szukamy najdłuższego wspólnego spójnego fragmentu (substring) ciągów $x_{1\dots n}$ i $y_{1\dots m}$. W odróżnieniu od LCS z wykładu fragment musi być ciągły.

Programowanie dynamiczne. Niech $D[i][j]$ to długość najdłuższego wspólnego *sufiksu* prefiksów $x_{1\dots i}$ i $y_{1\dots j}$ (czyli wspólnego fragmentu kończącego się dokładnie na x_i i y_j):

$$D[i][j] = \begin{cases} D[i-1][j-1] + 1, & x_i = y_j, \\ 0, & x_i \neq y_j, \end{cases} \quad D[0][j] = D[i][0] = 0.$$

Odpowiedzią jest $\max_{i,j} D[i][j]$ (indeksy początku odzyskujemy z pozycji maksimum: $i - k + 1, j - k + 1$).

Złożoność. $(n+1)(m+1)$ komórek, $\mathcal{O}(1)$ pracy każda: $\boxed{\mathcal{O}(mn)}$ czasu (pamięć $\mathcal{O}(m)$ przy trzymaniu dwóch wierszy).

Termin 3

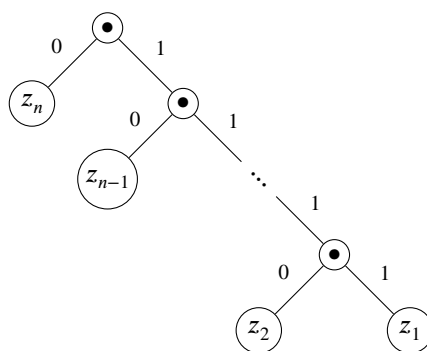
Zadanie 1 — kod Huffmana dla częstości będących potęgami dwójki (15 p.)

Alfabet A składa się ze znaków $z_1, z_2, \dots, z_n$. W pewnym tekście T znak z_i występuje 2^{i-1} razy dla wszystkich $i \in \{1, 2, \dots, n\}$. Znajdź kod Huffmana dla tekstu T .

Znak z_i występuje 2^{i-1} razy, $i = 1, \dots, n$ (częstości $1, 2, 4, \dots, 2^{n-1}$). Kluczowa obserwacja:

$$\sum_{j=1}^{i-1} 2^{j-1} = 2^{i-1} - 1 < 2^{i-1},$$

więc waga węzła scalającego pierwsze $i - 1$ znaków jest mniejsza od częstości kolejnego znaku z_i . Dlatego Huffman buduje „grzebień”: najpierw scala z_1 z z_2 , wynik z z_3 , ten z z_4 , itd.



Rysunek 1: Drzewo Huffmana dla częstości 2^{i-1} — „grzebień”.

Słowa kodowe i długości. Przyjmując konwencję „lewo = 0, prawo = 1” z rysunku:

$$z_n : 0, \quad z_{n-1} : 10, \quad z_{n-2} : 110, \quad \dots, \quad z_2 : \underbrace{1 \dots 1}_{n-2} 0, \quad z_1 : \underbrace{1 \dots 1}_{n-1}.$$

Długości: $\ell(z_i) = n - i + 1$ dla $i \geq 2$ oraz $\ell(z_1) = n - 1$ (oba najgłębsze znaki z_1, z_2 na poziomie $n - 1$). Sprawdzenie Krafta: $2 \cdot 2^{-(n-1)} + \sum_{i=3}^n 2^{-(n-i+1)} = 2^{-(n-2)} + (1 - 2^{-(n-2)}) = 1$.

Zadanie 2 — rozdział czekolady (15 p.)

Kostka czekolady o wadze $2n$ dkg spadła na podłogę i połamano się na wiele kawałków, z których żaden nie waży więcej niż 1 dkg. Zaprojektuj algorytm o liniowej złożoności obliczeniowej, który rozdziela te kawałki pomiędzy n dzieci tak, że każde dziecko dostaje kawałki o sumarycznej wadze co najmniej 1 dkg. Udowodnij, że Twój algorytm jest poprawny.

Kostka waży $2n$ dkg, połamana na kawałki, każdy ≤ 1 dkg; rozdzielamy je n dzieciom tak, by każde dostało ≥ 1 dkg.

Algorytm (liniowy). Przeglądamy kawałki w dowolnej ustalonej kolejności. Dla dziecka $j = 1, \dots, n-1$ dokładamy kolejne kawałki, dopóki jego suma nie osiągnie ≥ 1 dkg, po czym przechodzimy do dziecka $j+1$. Wszystkie pozostałe kawałki oddajemy dziecku n . Każdy kawałek dotykamy raz: czas $\mathcal{O}(\#\text{kawałków})$, liniowy.

Poprawność. Gdy dziecko $j < n$ kończymy obsługiwać, jego suma t_j spełnia $1 \leq t_j < 2$: przed ostatnim kawałkiem było < 1 , a kawałek dokłada ≤ 1 . Stąd pierwszych $n-1$ dzieci zużywa $\sum_{j < n} t_j < 2(n-1)$ dkg, więc dla dziecka n zostaje

$$R = 2n - \sum_{j < n} t_j > 2n - 2(n-1) = 2 \geq 1 \text{ dkg.}$$

Zatem każde z n dzieci dostaje ≥ 1 dkg — algorytm nie „zacina się” przed obsłużeniem wszystkich. $\square$

Zadanie 3 — stacje przekaznikowe (DP) (15 p.)

Firma „Głuchy Telefon” buduje stacje przekaznikowe wzdłuż autostrady A44 o długości M km. Jest n możliwych lokalizacji, leżących w odległościach $m_1 < m_2 < \dots < m_n$ od początku autostrady (wszystkie przy autostradzie). Koszt budowy stacji w lokalizacji i -tej to $k_i > 0$, $i \in \{1, \dots, n\}$. Stacje trzeba wybudować tak, by z każdego punktu autostrady było nie dalej niż k km do najbliższej stacji. Podaj efektywny algorytm programowania dynamicznego obliczający najmniejszy całkowity koszt inwestycji i oszacuj jego złożoność obliczeniową.

Autostrada $[0, M]$, możliwe lokalizacje w $m_1 < \dots < m_n$, koszty $k_i > 0$. Stacja w m_i obsługuje punkty w zasięgu k , czyli przedział $[m_i - k, m_i + k]$. Wybieramy podzbiór lokalizacji o minimalnym koszcie, którego przedziały pokrywają całe $[0, M]$.

Programowanie dynamiczne. Niech $D[i]$ to minimalny koszt zbioru stacji, w którym stacja i jest najdalej wysuniętą wybraną stacją, a odcinek $[0, m_i + k]$ jest w pełni pokryty (bez luk). Wtedy

$$D[i] = k_i + \begin{cases} 0, & m_i - k \leq 0 \quad (\text{stacja } i \text{ pokrywa początek}), \\ \min_{\substack{j < i \\ m_j + k \geq m_i - k}} D[j], & \text{w przeciwnym razie,} \end{cases}$$

gdzie warunek $m_j + k \geq m_i - k$ (tj. $m_i - m_j \leq 2k$) gwarantuje, że zasięg poprzedniej stacji j styka się z zasięgiem i — nie powstaje luka. Gdy oba przypadki są możliwe, bierzemy mniejszy; gdy żaden — $D[i] = \infty$. Odpowiedzią jest

$$\min_{i: m_i + k \geq M} D[i]$$

(jeśli zbiór jest pusty, pokrycie jest niewykonalne).

Złożoność. Dla każdego i minimalizujemy po $j < i$: $\mathcal{O}(n^2)$. Ponieważ lokalizacje są posortowane, a warunek $m_i - m_j \leq 2k$ jest monotoniczny, można utrzymywać minimum przesuwającym oknem i zejść do $\boxed{\mathcal{O}(n)}$ (po $\mathcal{O}(n \log n)$ na sortowanie, jeśli dane nieposortowane).

Zadanie 4 — obiekt majoryzujący (dziel i zwyciężaj) (15 p.)

Tablica A zawiera n obiektów, które nie są liczbami (np. małe pliki tekstowe). Nie jest dla nich zdefiniowana relacja $>$, ale zdefiniowana jest równość (sprawdzana w czasie stałym), oznaczająca identyczność obiektów. Mówimy, że tablica A ma *obiekt majoryzujący* M , jeśli więcej niż połowa obiektów jest równa M . Podaj algorytm typu „dziel i zwyciężaj” o złożoności $\mathcal{O}(n \log n)$, który rozstrzyga, czy A zawiera obiekt majoryzujący, i jeśli tak — zwraca go.

Tablica $A[1 \dots n]$ obiektów z samą relacją równości (sprawdzaną w $\mathcal{O}(1)$); M majoryzuje, gdy występuje $> n/2$ razy.

Algorytm.

- **Baza:** dla $n = 1$ zwróć ten jeden obiekt.
- **Dziel:** podziel A na połowy L, R i rekurencyjnie wyznacz ich kandydatów na majoryzujący: c_L, c_R (lub „brak”).
- **Połącz:** jeśli obiekt majoryzuje całe A , to majoryzuje co najmniej jedną połowę (gdyby w obu występował $\leq$ połowy, łącznie miałyby $\leq n/2$). Wystarczy więc sprawdzić c_L i c_R : przeglądając A raz, zlicz wystąpienia każdego z nich i zwróć tego, który występuje $> n/2$ razy (jeśli żaden — „brak”).

Złożoność. $T(n) = 2T(n/2) + \mathcal{O}(n) = \boxed{\mathcal{O}(n \log n)}$.

Uwaga. Ten sam problem (przy samej równości) rozwiązuje liniowo głosowanie Boyera–Moore’a, ale zadanie wymaga schematu „dziel i zwyciężaj”.